



Chapter 1: Introduction

A database is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are managed by Database Management Systems (DBMS), which allow users to create, read, update, and delete data efficiently.

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use

A database is a system used to store, organize, and manage data. It allows for easy retrieval, updating, and manipulation of data. Databases are used in various applications, from websites and business systems to mobile apps.

There are two main types of databases:

Relational Databases: Data is stored in tables with rows and columns. Examples: MySQL, PostgreSQL, SQL Server.

Non-Relational Databases: Data is stored in more flexible formats, like documents or key-value pairs. Examples: MongoDB, Redis.

Databases can be used for storing anything, such as customer records, inventory, transactions, and more. They help ensure data integrity, security, and efficient access.



Database Applications Examples

- Enterprise Information
 - Sales: customers, products, purchases
 - Accounting: payments, receipts, assets
 - Human Resources: Information about employees, salaries, payroll taxes.
- Manufacturing: management of production, inventory, orders, supply chain.
- Banking and finance
 - customer information, accounts, loans, and banking transactions.
 - Credit card transactions
 - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data
- Universities: registration, grades



Database Applications Examples (Cont.)

- Airlines: reservations, schedules
- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
 - Online retailers: order tracking, customized recommendations
 - Online advertisements
- Document databases
- Navigation systems: For maintaining the locations of various places of interest along with the exact routes of roads, train systems, buses, etc.



Purpose of Database Systems

The purpose of database systems is to efficiently store, manage, and retrieve data in a structured way, which leads to:

Data Storage: Safely store large amounts of data in an organized manner.

Data Retrieval: Quickly retrieve data when needed, making it easy to search, filter, and analyze.

Data Integrity: Ensure that the data is accurate and consistent by enforcing rules and constraints.

Data Security: Protect data from unauthorized access through authentication, authorization, and encryption.

Data Sharing: Allow multiple users and applications to access and modify data simultaneously, ensuring collaboration.



Purpose of Database Systems (Cont.)

Data Backup & Recovery: Enable backup of data to prevent loss and provide recovery options in case of failure.

Data Redundancy Elimination: Minimize duplication of data through normalization to optimize storage and reduce inconsistencies.

Efficient Management: Streamline operations like inserting, updating, and deleting data with minimal effort and time.

These systems help maintain the performance, reliability, and scalability of applications across different industries.



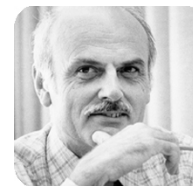
Data Models

- Data models are abstract representations of how data is structured, organized, and manipulated in a database or information system. A collection of tools for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- Relational model
- Entity-Relationship data model (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semi-structured data model (XML)
- Other older models:
 - Network model
 - Hierarchical model



Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model



Ted Codd
Turing Award 1981

Columns

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows

(a) The *instructor* table



A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

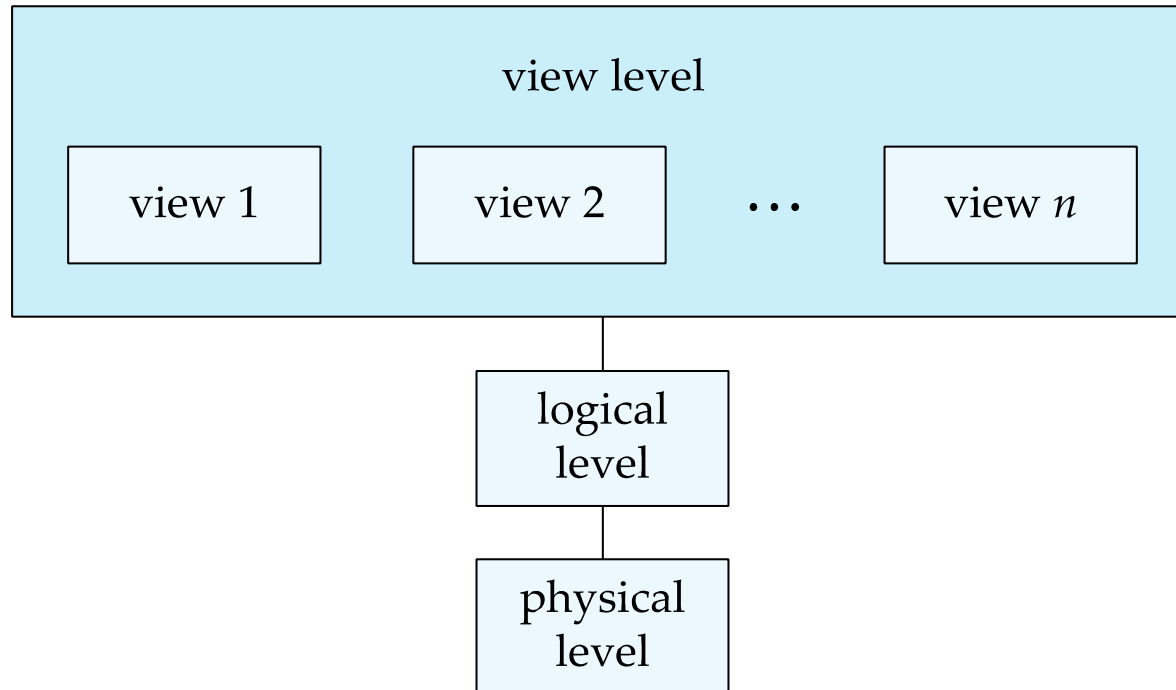
<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table



View of Data

An architecture for a database system





Instances and Schemas

- A schema is a structured framework that defines how data is organized in a database, including tables, fields, data types, and relationships between tables.
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Physical schema** – the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable



Physical Data Independence

- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema

Key Points:

What it Means: Changes in how data is stored (e.g., indexing, file structures, or storage location) do not affect how data is accessed at the logical level.

Why It's Important: It allows improvements in storage efficiency and performance without modifying application code.

Example: If a database administrator moves data from an HDD to an SSD or changes the indexing strategy, queries and applications remain unchanged.

This independence makes database systems more flexible, scalable, and easier to maintain.



Data Definition Language (DDL)

Data Definition Language (DDL) is a set of SQL commands used to define and manage the structure of a database. It deals with creating, modifying, and deleting database objects like tables, indexes, and schemas

Key DDL Commands

CREATE – Creates a new database or table

ALTER – Modifies an existing table (e.g., adding a column)

DROP – Deletes a table or database permanently.

TRUNCATE – Deletes all records from a table but keeps its structure

DDL commands directly affect the database structure and require commit automatically (cannot be rolled back).

A Data Dictionary is a centralized repository that contains information about the structure, organization, and properties of data in a database. It acts as a reference guide for database administrators and developers.

Contents of a Data Dictionary

Table Names – List of all tables in the database.

Column Names & Data Types – Information about columns, their data types, and constraints.

Primary & Foreign Keys – Relationships between tables.

Indexes – Details about indexes used for performance optimization.

Constraints – Rules like NOT NULL, UNIQUE, and CHECK.

User Permissions – Access control and security settings.

Stored Procedures & Triggers – Definitions of procedures and automated actions.



Data Manipulation Language (DML)

Data Manipulation Language (DML) is a subset of SQL used to interact with and manipulate data in a database. Unlike DDL, which defines the structure, DML focuses on modifying data within tables. DML also known as query language.

- Two classes of languages
 - **Pure** – used for proving properties about computational power and for optimization
 - Relational Algebra
 - Tuple relational calculus
 - Domain relational calculus
 - **Commercial** – used in commercial systems
 - SQL is the most widely used commercial language



Data Manipulation Language (Cont.)

- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify what data are needed and how to get those data.
 - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.

Key DML Commands

INSERT – Adds new records to a table.

UPDATE – Modifies existing records.

DELETE – Removes records from a table.

SELECT – Retrieves data from a table



SQL Query Language

- SQL (Structured Query Language) is a non-procedural language used for managing and manipulating relational databases. It allows users to perform various operations such as querying, inserting, updating, and deleting data.
- Example to find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```
- SQL is **NOT** a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database



Database Access from Application Program

- Non-procedural query languages such as SQL are not as powerful as a universal Turing machine.
- SQL does not support actions such as input from users, output to displays, or communication over the network.
- Such computations and actions must be written in a **host language**, such as C/C++, Java or Python, with embedded SQL queries that access the data in the database.
- **Application programs** -- are programs that are used to interact with the database in this fashion.



Database Design

The process of designing the general structure of the database:

- **Logical Design** – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **Physical Design** – Deciding on the physical layout of the database



Database Engine

- A Database Engine is the core software component of a database management system (DBMS) that handles data storage, management, and retrieval. It performs essential functions like data insertion, updating, deletion, and querying. A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be divided into
 - The storage manager,
 - The query processor component,
 - The transaction management component.



Storage Manager

- A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - Interaction with the OS file manager
 - Efficient storing, retrieving and updating of data\
- The storage manager components include:
 - Authorization and integrity manager
 - Transaction manager
 - File manager
 - Buffer manager



Storage Manager (Cont.)

- The storage manager implements several data structures as part of the physical system implementation:
 - Data files -- store the database itself
 - Data dictionary -- stores metadata about the structure of the database, in particular the schema of the database.
 - Indices -- can provide fast access to data items. A database index provides pointers to those data items that hold a particular value.



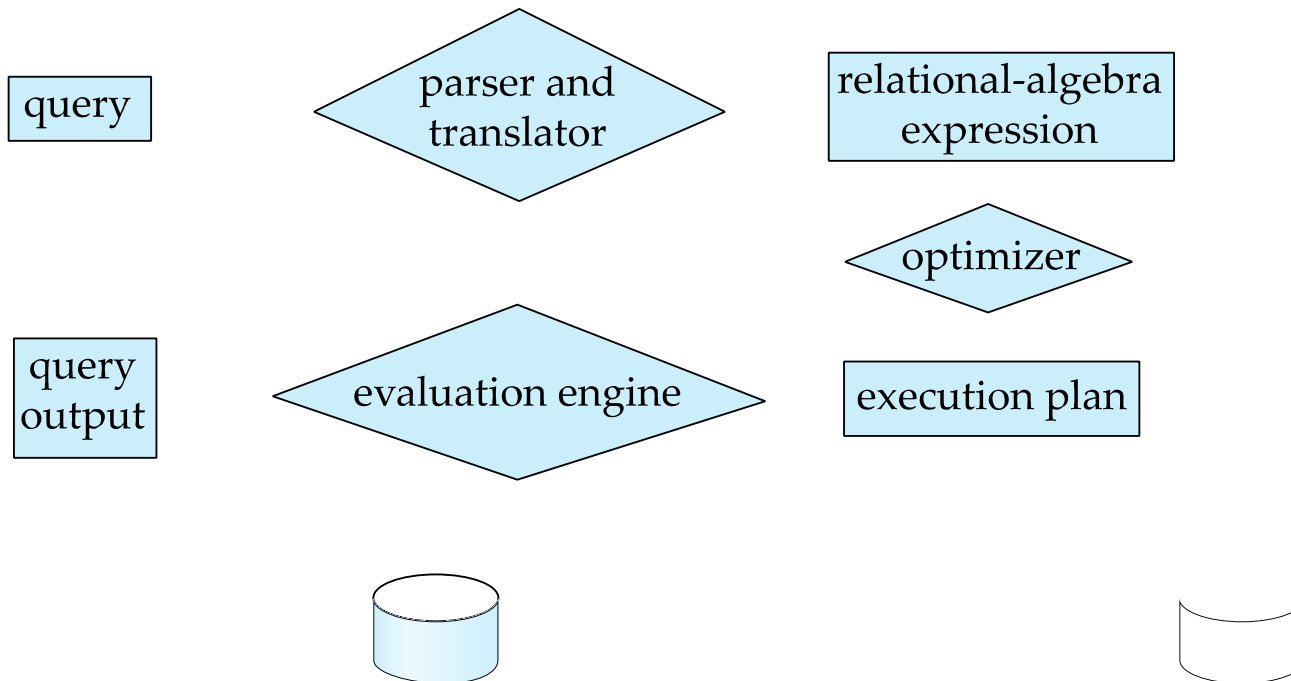
Query Processor

- The query processor components include:
 - **DDL interpreter** -- interprets DDL statements and records the definitions in the data dictionary.
 - **DML compiler** -- translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
 - The DML compiler performs query optimization; that is, it picks the lowest cost evaluation plan from among the various alternatives.
 - **Query evaluation engine** -- executes low-level instructions generated by the DML compiler.



Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Transaction Management

- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.



Database Architecture

1. Centralized Databases

All data and processing are centralized on a single machine with shared memory.

Typically involve one or a few processing cores.

2. Client-Server Databases

A server machine (or machines) processes and executes database requests from multiple client machines.

Clients interact with the server over a network.

3. Parallel Databases

Designed for high-performance processing with multiple processors.

Shared Memory: Multiple cores/processors with shared access to memory.

Shared Disk: Processors share access to the same disk storage.

Shared Nothing: Each processor has its own memory and disk, improving scalability and reducing contention.

4. Distributed Databases

Data is distributed across multiple geographic locations.

Can have schema/data heterogeneity—data across sites may have different schemas or formats.

Allows for local data processing with global data accessibility.



Database Applications

Database applications are usually partitioned into two or three parts

▪ Two-tier architecture --

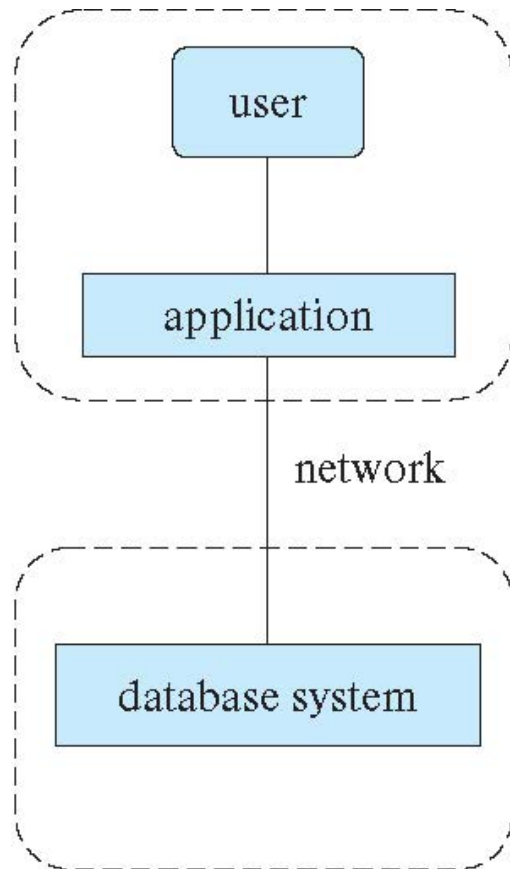
- 1.The application is on the client machine, which directly connects to the database on the server.
- 2.The client requests data or functions, and the server processes them

▪ Three-tier architecture --

- 1.The client (user's device) is only for the interface and does not connect directly to the database.
- 2.Instead, the client communicates with an application server (middle layer) that handles all the business logic.
- 3.The application server then connects to the database to retrieve or update data as needed.



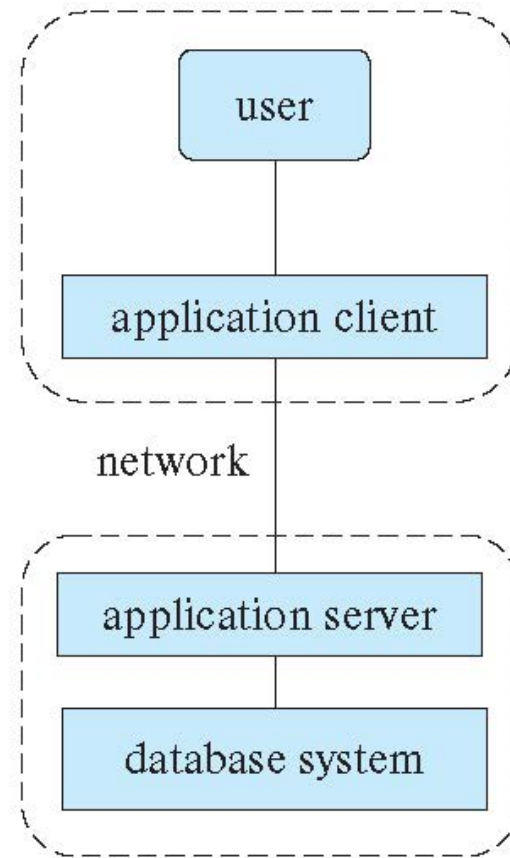
Two-tier and three-tier architectures



(a) Two-tier architecture

client

server



(b) Three-tier architecture



Database Users

There are four different types of database-system users

- **Naive users** -- unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- **Application programmers** -- are computer professionals who write application programs.
- **Sophisticated users** -- interact with the system without writing programs
 - using a database query language or by
 - using tools such as data analysis software.
- **Specialized users** -- write specialized database applications that do not fit into the traditional data-processing framework. For example, CAD, graphic data, audio, video.



Database Administrator

A person who has central control over the system is called a **database administrator (DBA)**, whose functions are:

- Schema definition
- Storage structure and access-method definition
- Schema and physical-organization modification
- Granting of authorization for data access
- Routine maintenance
- Periodically backing up the database
- Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required
- Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users



History of Database Systems

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - Would win the ACM Turing Award for this work
 - IBM Research begins System R prototype
 - UC Berkeley (Michael Stonebraker) begins Ingres prototype
 - Oracle releases first commercial relational database
 - High-performance (for the era) transaction processing



History of Database Systems (Cont.)

- 1980s:
 - Research relational prototypes evolve into commercial systems
 - SQL becomes industrial standard
 - Parallel and distributed database systems
 - Wisconsin, IBM, Teradata
 - Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce



History of Database Systems (Cont.)

- 2000s
 - Big data storage systems
 - Google BigTable, Yahoo PNuts, Amazon,
 - “NoSQL” systems.
 - Big data analysis: beyond SQL
 - Map reduce and friends
- 2010s
 - SQL reloaded
 - SQL front end to Map Reduce systems
 - Massively parallel database systems
 - Multi-core main-memory databases



End of Chapter 1